Terraform challenges-02

You are tasked with creating a codebase for a microservice deployment on a cloud environment, using Terraform. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

Requirements

Service Details

- **The service will be a simple web server that returns a static page.**
- **You can use apache2 or nginx to serve a simple HTML file.**
- **The service should be containerized and stored in a Docker registry.**
- **The service should be deployed on a single VM instance as a docker container.**
- **The image should be pulled from the registry specified earlier.**
- **The service should not be accessible from the internet but only through a Load Balancer, except for SSH access from specific IP addresses.**

Deliverables

You are to provide the following as part of your solution:

1. **Dockerfile to build the image.**
2. **Terraform codebase to deploy the following:**
    - **Docker registry**
    - **VPC and all required networking resources**
    - **EC2 instance**
    - **Load Balancer**

Notes

- **Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.**
- **To push the image to the Docker registry, build the image locally before running the Terraform code.**

- **Once the image registry is created, use Terraform to push the local image to the registry.**
- **Similarly, use Terraform to pull the image from the registry and run it on the VM instance once it is created.**
- **For SSH access:**
    - **Use a locally created key.**
    - **Add it to the cloud provider key storage.**
    - **Attach it to the instance.**
- **Use Terraform best practices for file structure and code organization.**
    - **Use variables, locals, modules, etc. when appropriate.**
- **Include all tfvars, scripts, and other helper files used to deploy the application.**

Objective

To deploy a containerized static web application on AWS using Terraform. The Docker image is stored in Amazon ECR and deployed on an EC2 instance. The application is accessible through an Application Load Balancer (ALB), while direct application access to the EC2 instance is restricted.

Implementation Steps

1. Create the Static Web Application

1. Create an app folder in the project directory.
2. Create an index.html file containing the static web page.
3. Create a Dockerfile using the Nginx Alpine base image.
4. Copy index.html into the Nginx default web directory.
5. Expose port 80 in the Dockerfile.

2. Prepare the Terraform Project Structure

1. Create a project folder named microservices_terraform.
2. Create separate Terraform files for provider configuration, variables, locals, ECR, networking, security groups, EC2, ALB and outputs.
3. Create a scripts folder for the image-push PowerShell script and EC2 user-data template.
4. Configure variables and values in terraform.tfvars.
5. Configure common resource tags using locals.tf.

3. Configure AWS Provider

1. Configure the AWS provider in provider.tf.
2. Set the AWS region to us-east-1.
3. Specify the required AWS provider version as 6.70.0.
4. Configure the required Terraform providers in versions.tf.

4. Configure the Existing EC2 Key Pair

1. Use the existing EC2 key pair registered in AWS.
2. Configure an aws_key_pair data source in ec2.tf.
3. Reference the existing key pair name in the EC2 instance configuration.
4. Keep the corresponding private .pem file locally for SSH access.

5. Build the Docker Image

1. Start Docker Desktop.
2. Open PowerShell in the project directory.
3. Build the Docker image for the Linux AMD64 platform.

   docker build --platform linux/amd64 -t static-web:1.0 ./app
4. Verify the image.

   docker images

6. Create an Amazon ECR Repository

1. Configure an Amazon ECR repository using Terraform.
2. Configure image tag mutability.
3. Enable image scanning on image push.
4. Configure AES256 encryption.
5. Use the ECR repository URL for storing the Docker image.

7. Push the Docker Image to Amazon ECR

1. Create a PowerShell script named push-image.ps1.
2. Retrieve an authentication token using the AWS CLI.
3. Authenticate Docker with Amazon ECR.
4. Tag the local Docker image with the ECR repository URL.
5. Push the image to the ECR repository.
6. Configure a Terraform null_resource with a local-exec provisioner to execute the PowerShell script.
7. Configure triggers using the Dockerfile hash, HTML file hash, image tag and repository URL.

8. Create VPC and Networking

1. Create a custom VPC with CIDR block 10.0.0.0/16.
2. Create an Internet Gateway and attach it to the VPC.
3. Create two public subnets in different Availability Zones.
4. Enable automatic public IP assignment for the public subnets.
5. Create a custom public route table.
6. Add a default route (0.0.0.0/0) through the Internet Gateway.
7. Associate both public subnets with the route table.

9. Configure Security Groups

1. Create an ALB security group.
2. Allow inbound HTTP traffic on port 80 from the internet.
3. Create an EC2 security group.
4. Allow SSH access on port 22 only from the specified public IP addresses.
5. Allow HTTP traffic on port 80 only from the ALB security group.
6. Configure outbound traffic for required application and image downloads.

10. Configure IAM Role for EC2

1. Create an IAM role for EC2.
2. Configure the EC2 service as the trusted entity.
3. Attach the AmazonEC2ContainerRegistryReadOnly managed policy.
4. Create an IAM instance profile.
5. Attach the instance profile to the EC2 instance.

11. Create the EC2 Instance

1. Retrieve the latest Ubuntu 24.04 AMI using a Terraform data source.
2. Configure the EC2 instance type as t3.micro.
3. Select the first public subnet.
4. Attach the EC2 security group.
5. Associate a public IP address.
6. Attach the existing EC2 key pair.
7. Attach the IAM instance profile.
8. Configure the root EBS volume.
9. Add the required resource tags.

12. Configure EC2 User Data

1. Create a user-data template named user_data.sh.tftpl.
2. Update the Ubuntu package repositories.
3. Install Docker and AWS CLI.
4. Enable and start the Docker service.
5. Authenticate Docker with Amazon ECR using the EC2 IAM role.
6. Pull the Docker image from the ECR repository.
7. Create and start the application container.
8. Map EC2 port 80 to container port 80.
9. Configure the container to restart automatically.

13. Create the Application Load Balancer

1. Create an internet-facing Application Load Balancer.
2. Select both public subnets.
3. Attach the ALB security group.
4. Create a target group using HTTP protocol on port 80.
5. Configure the health check path as /.
6. Set the expected health check response code to 200.
7. Register the EC2 instance with the target group.
8. Create an HTTP listener on port 80.
9. Configure the listener to forward requests to the target group.

14. Configure Terraform Outputs

1. Create an output for the ECR repository URL.
2. Create an output for the EC2 instance ID.
3. Create an output for the EC2 public IP.
4. Create an output for the ALB DNS name.
5. Create an application URL using the ALB DNS name.

15. Initialize and Validate Terraform

1. Open PowerShell in the project directory.
2. Initialize Terraform providers.

   terraform init
3. Format the Terraform configuration files.

terraform fmt -recursive

4. Validate the configuration.

   terraform validate

5. Review the infrastructure execution plan.

   terraform plan

16. Deploy the Infrastructure

1. Execute the Terraform deployment.

   terraform apply

2. Review the execution plan.
3. Enter yes to approve the deployment.
4. Terraform creates the AWS infrastructure.
5. The local-exec provisioner authenticates Docker and pushes the image to ECR.
6. EC2 user data installs Docker, pulls the image and starts the container.

17. Verify the Application Deployment

1. Retrieve the Terraform outputs.

   terraform output

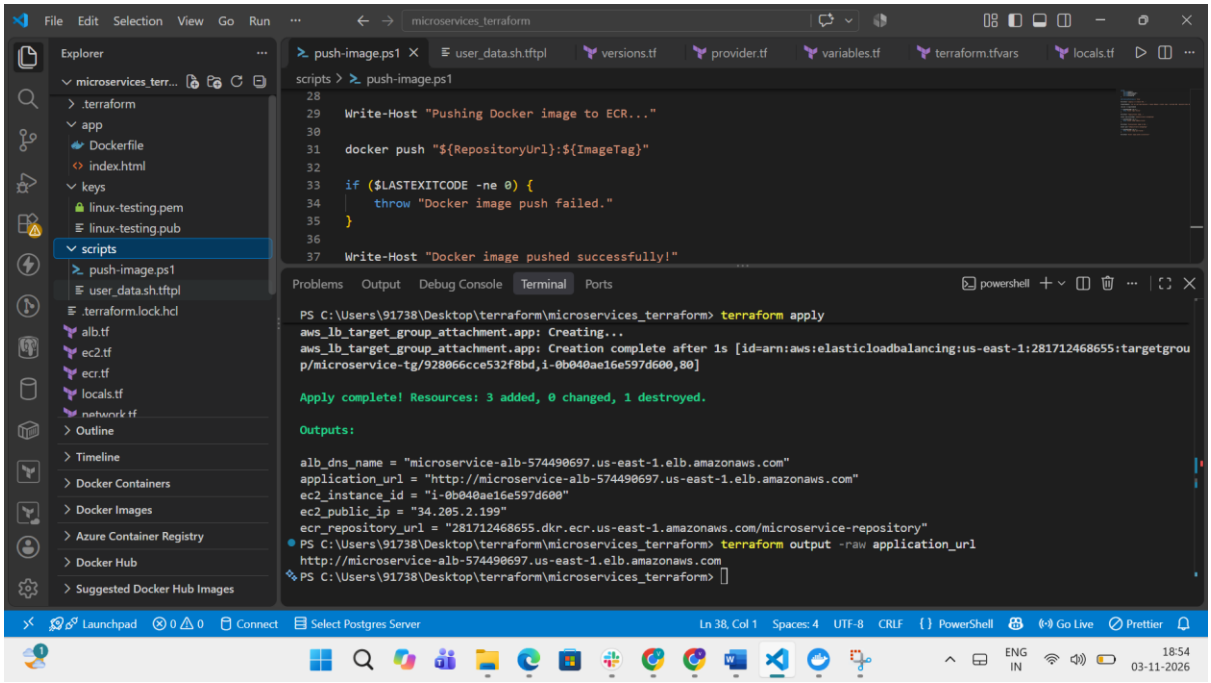
2. Get the ALB application URL.

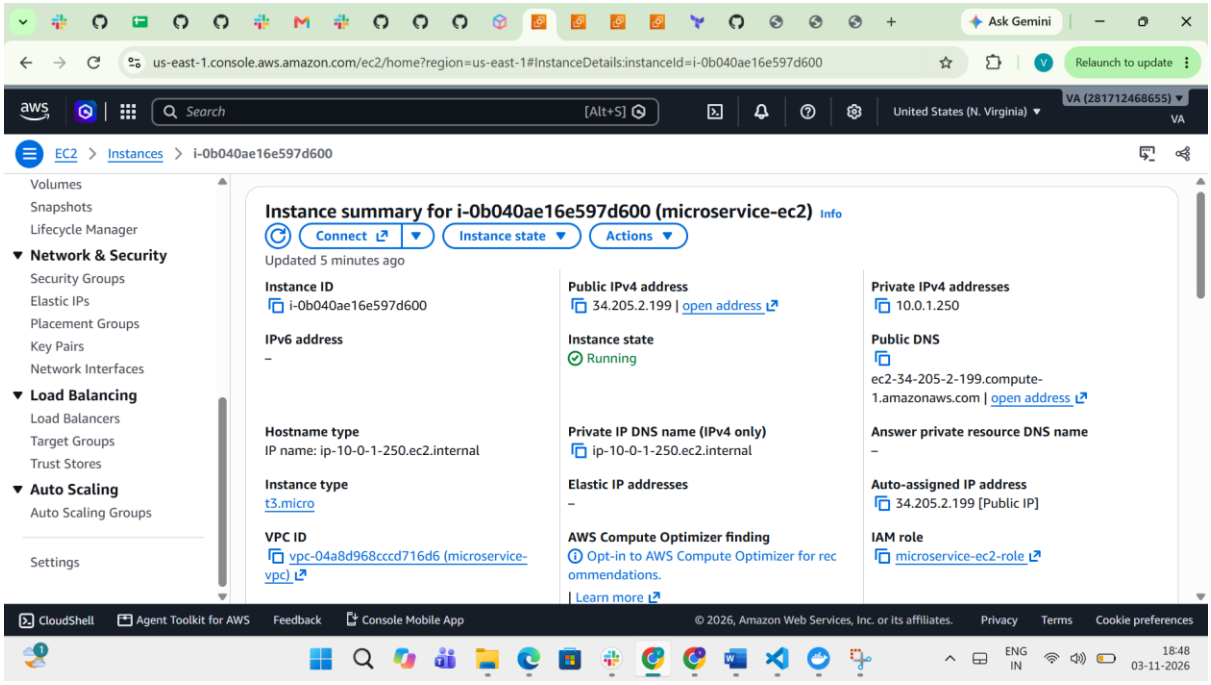
   terraform output -raw application_url

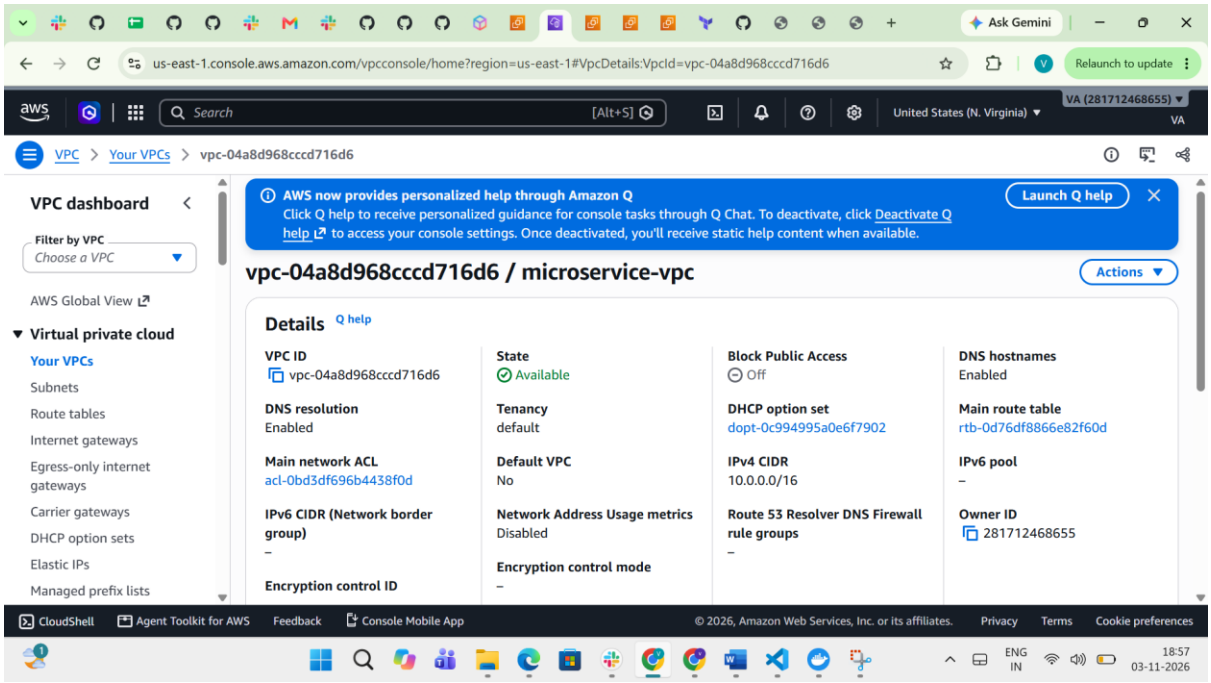
3. Open the ALB URL in a web browser.
4. Verify that the static web application is displayed.
5. Verify the EC2 instance is registered as healthy in the target group.
6. Verify the Docker container is running on EC2.

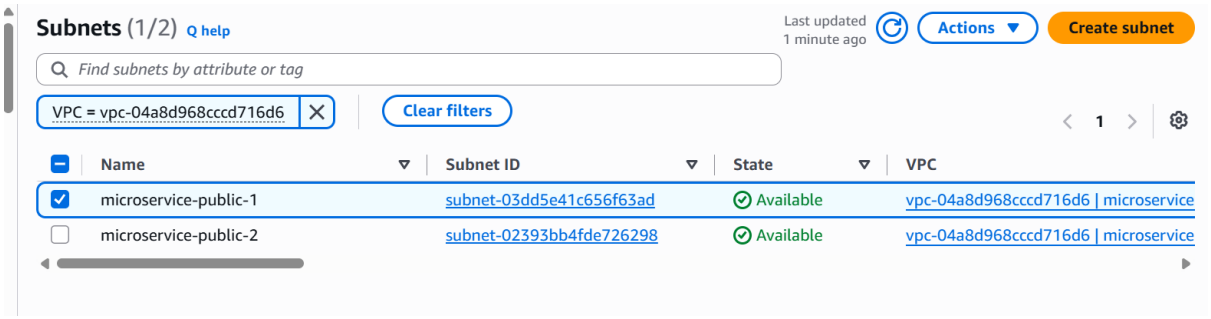
18. Verify Security Configuration

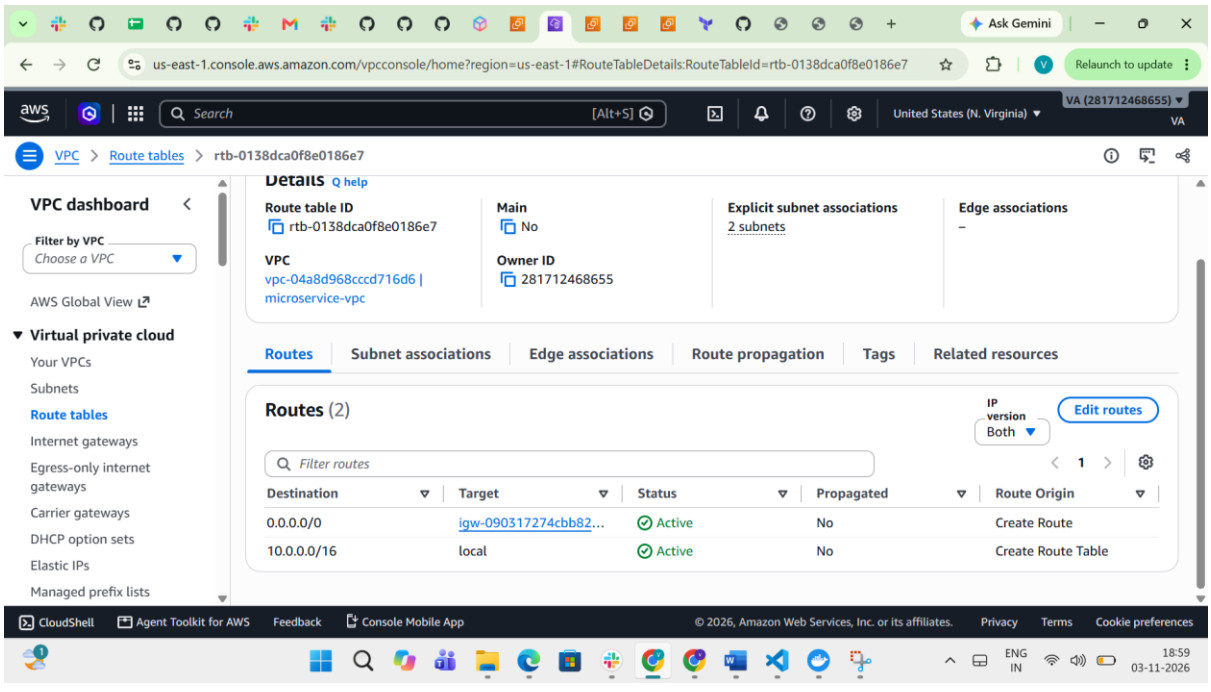
1. Access the application through the ALB DNS name.
2. Verify that direct HTTP access to the EC2 public IP is blocked.
3. Verify that SSH access is permitted only from the configured public IP addresses.
4. Verify that the EC2 security group accepts application traffic only from the ALB security group.


File Edit Selection View Go Run
microservices_terraform
Explorer
push-image.ps1
user_data.sh.tftpl
versions.tf
provider.tf
variables.tf
terraform.tfvars
locals.tf
scripts > push-image.ps1
Write-Host "Pushing Docker image to ECR..."
docker push "${RepositoryUrl}:${ImageTag}"
if ($LASTEXITCODE -ne 0) {
throw "Docker image push failed."
}
Write-Host "Docker image pushed successfully!"
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\91738\Desktop\terraform\microservices_terraform> terraform apply
aws_lb_target_group_attachment.app: Creating...
aws_lb_target_group_attachment.app: Creation complete after 1s [id=arn:aws:elasticloadbalancing:us-east-1:281712468655:targetgroup/microservice-tg/928066cce532f8bd,i-0b040ae16e597d600,80]
Apply complete! Resources: 3 added, 0 changed, 1 destroyed.
Outputs:
alb_dns_name = "microservice-alb-574490697.us-east-1.elb.amazonaws.com"
application_url = "http://microservice-alb-574490697.us-east-1.elb.amazonaws.com"
ec2_instance_id = "i-0b040ae16e597d600"
ec2_public_ip = "34.205.2.199"
ecr_repository_url = "281712468655.dkr.ecr.us-east-1.amazonaws.com/microservice-repository"
PS C:\Users\91738\Desktop\terraform\microservices_terraform> terraform output -raw application_url
http://microservice-alb-574490697.us-east-1.elb.amazonaws.com
PS C:\Users\91738\Desktop\terraform\microservices_terraform>
Launchpad
Connect
Select Postgres Server
Ln 38, Col 1
Spaces: 4
UTF-8
CRLF
PowerShell
Go Live
Prettier
ENG IN
18:54
03-11-2026


Ask Gemini
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-0b040ae16e597d600
Relaunch to update
Search
[Alt+S]
United States (N. Virginia)
VA (281712468655)
EC2 > Instances > i-0b040ae16e597d600
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Settings
Instance summary for i-0b040ae16e597d600 (microservice-ec2) Info
Connect
Instance state
Actions
Updated 5 minutes ago
Instance ID
i-0b040ae16e597d600
IPv6 address
–
Hostname type
IP name: ip-10-0-1-250.ec2.internal
Instance type
t3.micro
VPC ID
vpc-04a8d968cccd716d6 (microservice-vpc)
Public IPv4 address
34.205.2.199 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
ip-10-0-1-250.ec2.internal
Elastic IP addresses
–
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
Learn more
Private IPv4 addresses
10.0.1.250
Public DNS
ec2-34-205-2-199.compute-1.amazonaws.com | open address
Answer private resource DNS name
–
Auto-assigned IP address
34.205.2.199 [Public IP]
IAM role
microservice-ec2-role
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
ENG IN
18:48
03-11-2026


us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#VpcDetails:VpcId=vpc-04a8d968cccd716d6
VPC > Your VPCs > vpc-04a8d968cccd716d6
VPC dashboard
AWS now provides personalized help through Amazon Q
Launch Q help
vpc-04a8d968cccd716d6 / microservice-vpc
Actions
Details
VPC ID
vpc-04a8d968cccd716d6
State
Available
Block Public Access
Off
DNS hostnames
Enabled
DNS resolution
Enabled
Tenancy
default
DHCP option set
dopt-0c994995a0e6f7902
Main route table
rtb-0d76df8866e82f60d
Main network ACL
acl-0bd3df696b4438f0d
Default VPC
No
IPv4 CIDR
10.0.0.0/16
IPv6 pool
Network Address Usage metrics
Disabled
Route 53 Resolver DNS Firewall rule groups
Owner ID
281712468655
Encryption control ID
Encryption control mode
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Carrier gateways
DHCP option sets
Elastic IPs
Managed prefix lists
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Subnets (1/2)
Last updated 1 minute ago
Actions
Create subnet
Find subnets by attribute or tag
VPC = vpc-04a8d968cccd716d6
Clear filters
Name
Subnet ID
State
VPC
microservice-public-1
subnet-03dd5e41c656f63ad
Available
vpc-04a8d968cccd716d6 | microservice
microservice-public-2
subnet-02393bb4fde726298
Available
vpc-04a8d968cccd716d6 | microservice


us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#RouteTableDetails:RouteTableId=rtb-0138dca0f8e0186e7
VPC > Route tables > rtb-0138dca0f8e0186e7
Details
Route table ID
rtb-0138dca0f8e0186e7
Main
No
Explicit subnet associations
2 subnets
Edge associations
VPC
vpc-04a8d968cccd716d6 | microservice-vpc
Owner ID
281712468655
Routes
Subnet associations
Edge associations
Route propagation
Tags
Related resources
Routes (2)
Edit routes
IP version
Both
Filter routes
Destination
Target
Status
Propagated
Route Origin
0.0.0.0/0
igw-090317274cbb82...
Active
No
Create Route
10.0.0.0/16
local
Active
No
Create Route Table

Routes | Subnet associations | Edge associations | Route propagation | Tags | Related resources

Explicit subnet associations (2)

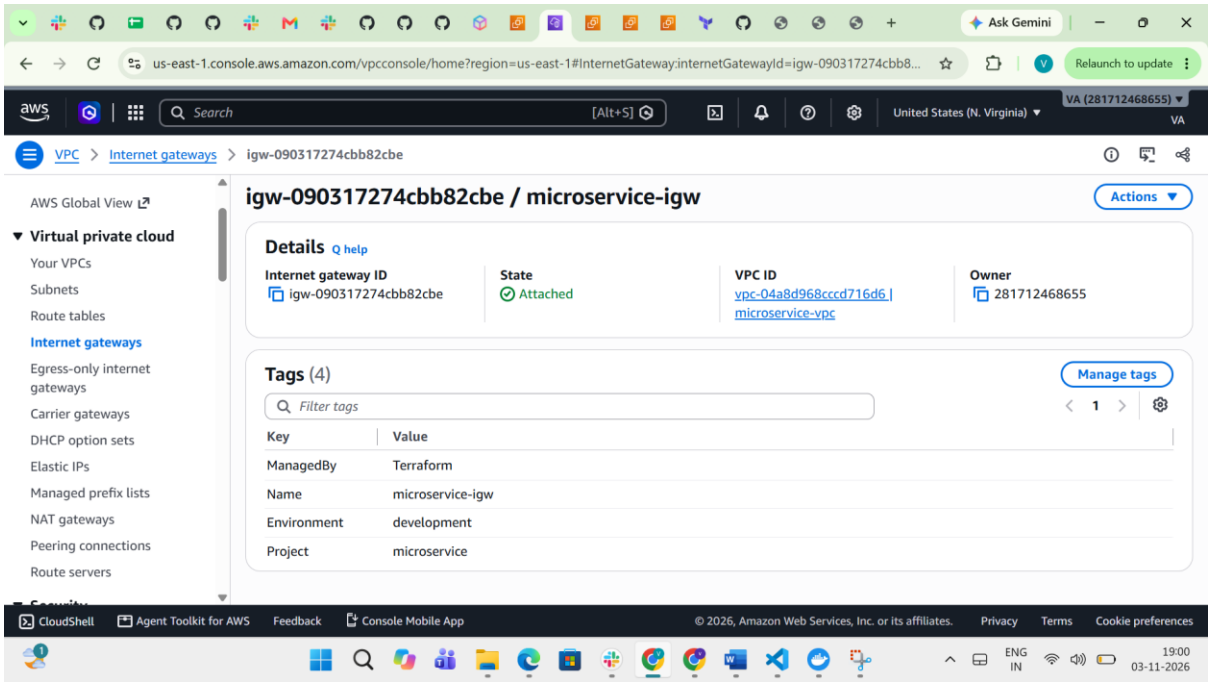
Edit subnet associations

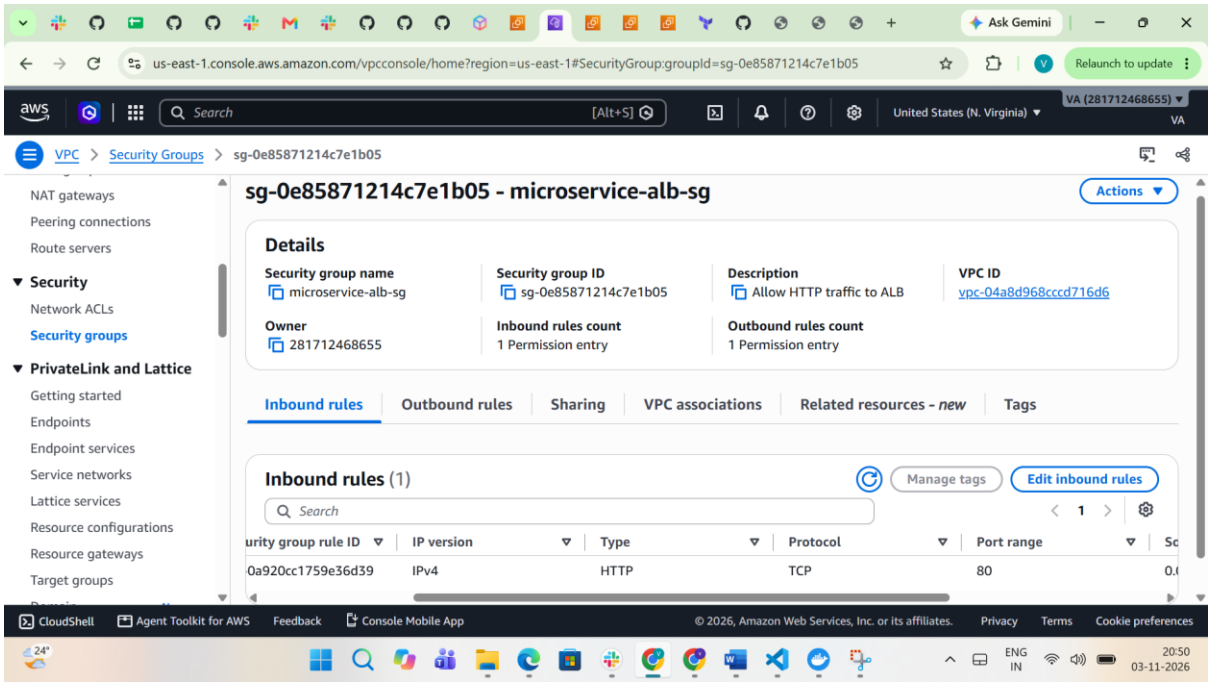
Find subnet association

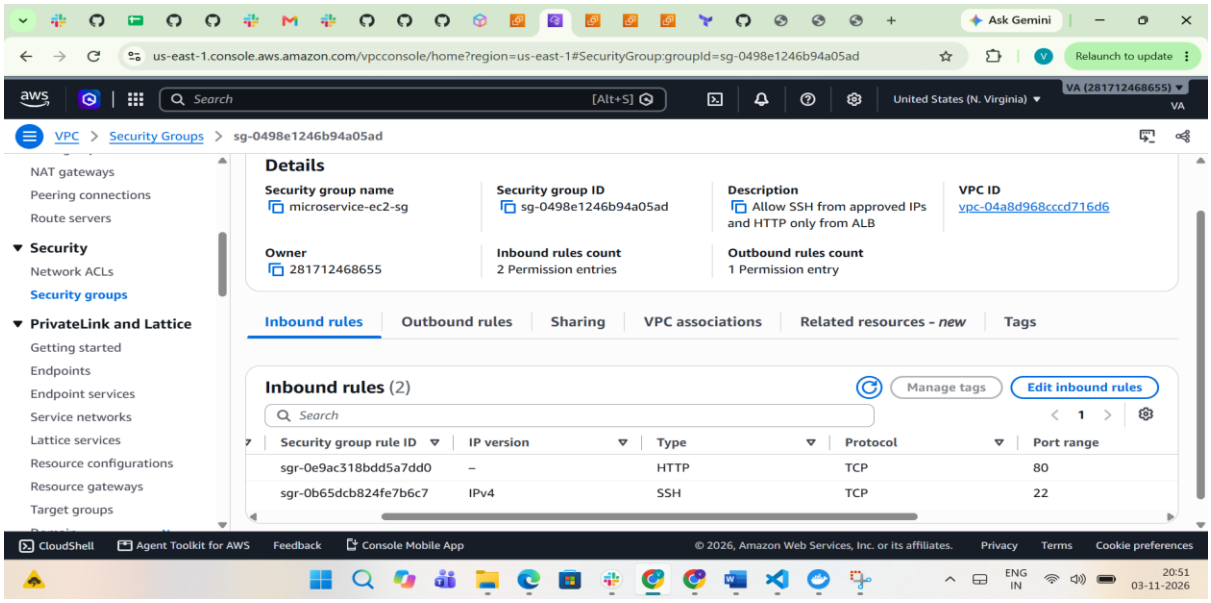
Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
microservice-public-1	subnet-03dd5e41c656f63ad	10.0.1.0/24	–
microservice-public-2	subnet-02393bb4fde726298	10.0.2.0/24	–

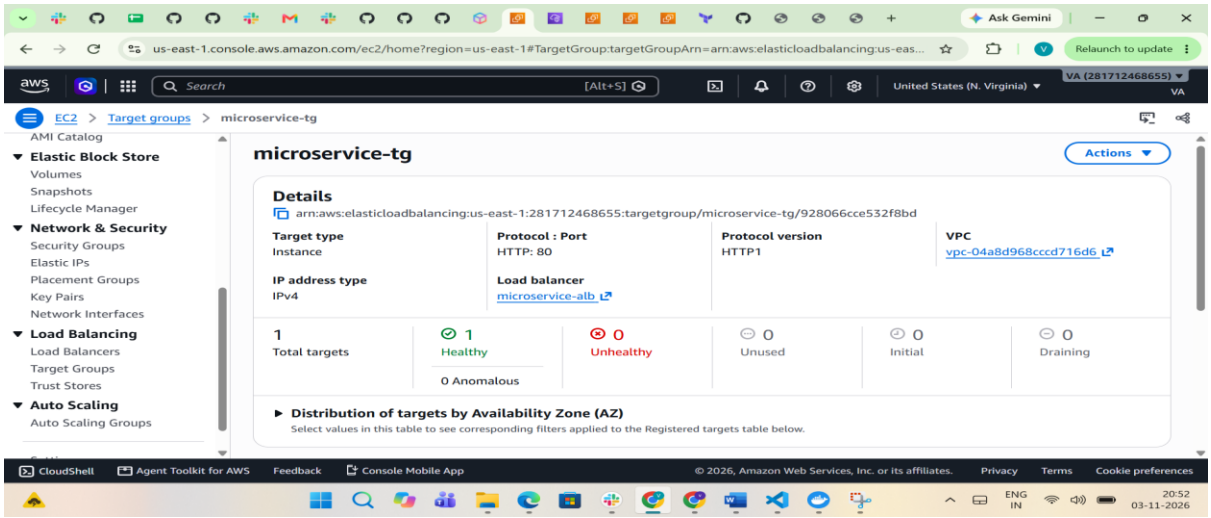
Subnets without explicit associations (0)

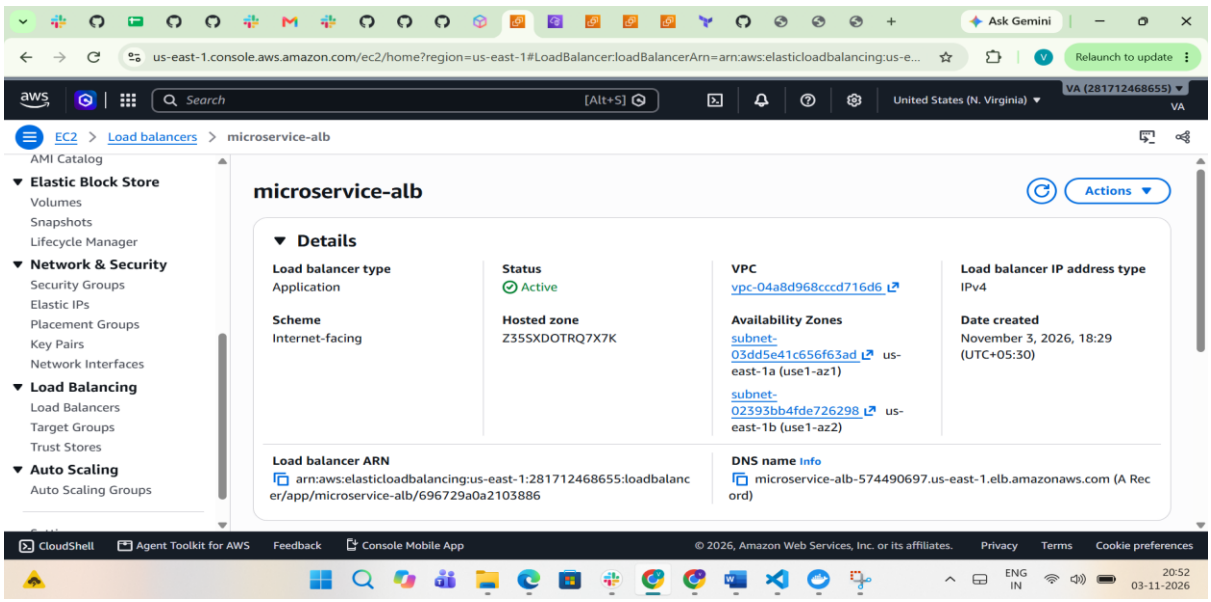
Edit subnet associations

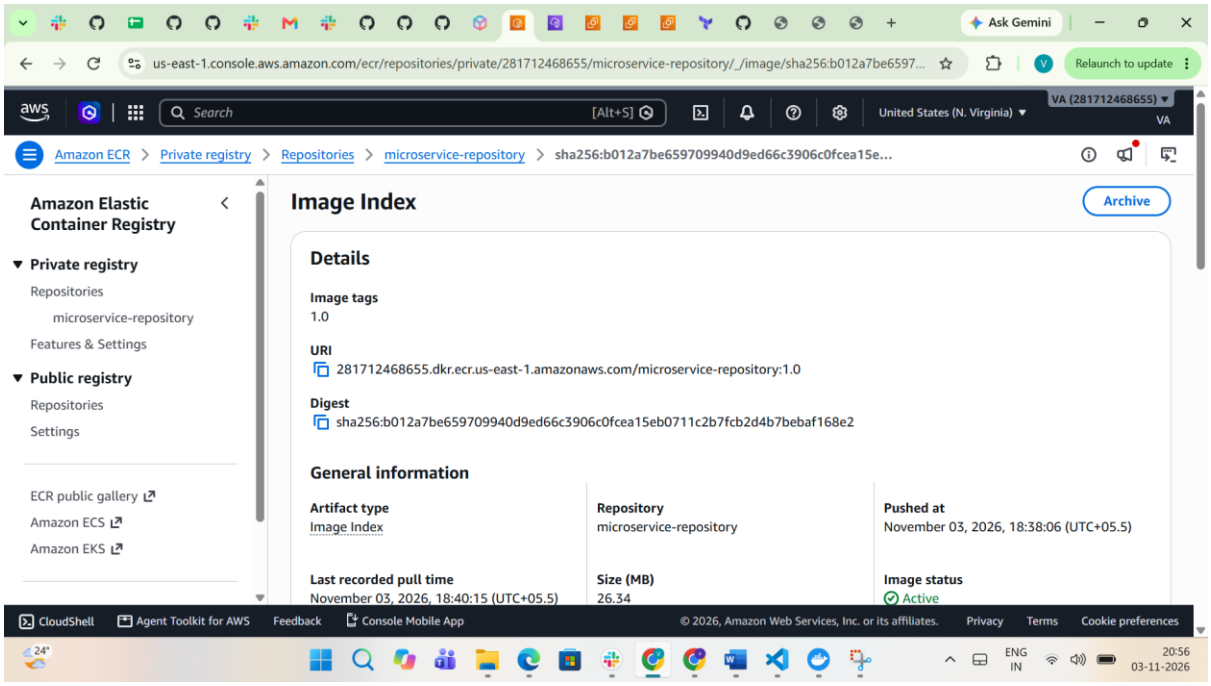


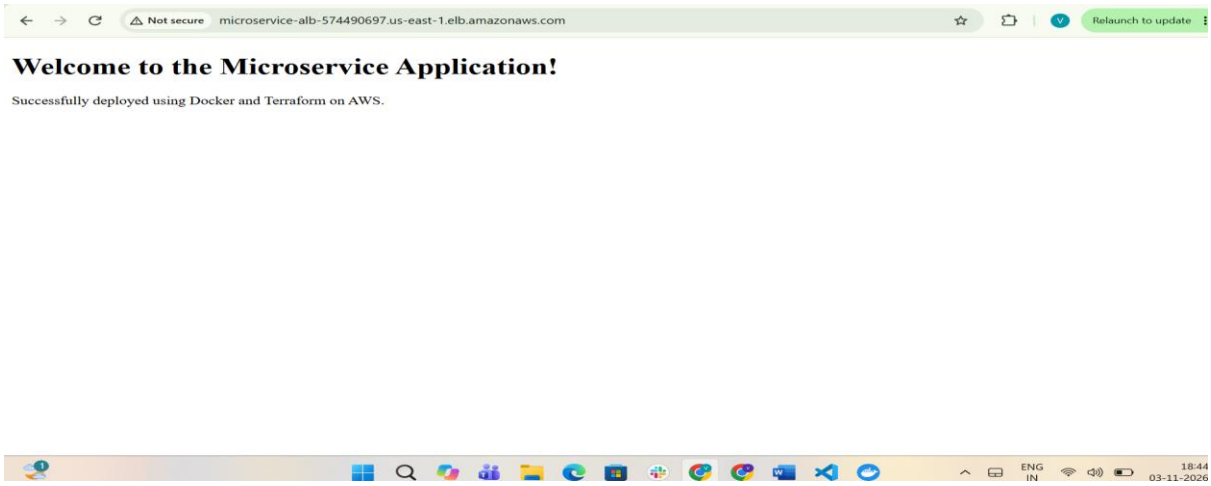



us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#SecurityGroup:groupId=sg-0498e1246b94a05ad
VPC > Security Groups > sg-0498e1246b94a05ad
Details
Security group name
microservice-ec2-sg
Security group ID
sg-0498e1246b94a05ad
Description
Allow SSH from approved IPs and HTTP only from ALB
VPC ID
vpc-04a8d968cccd716d6
Owner
281712468655
Inbound rules count
2 Permission entries
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (2)
Manage tags
Edit inbound rules
Security group rule ID
IP version
Type
Protocol
Port range
sgr-0e9ac318bdd5a7dd0
-
HTTP
TCP
80
sgr-0b65dcb824fe7b6c7
IPv4
SSH
TCP
22


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#TargetGroup:targetGroupArn=arn:aws:elasticloadbalancing:us-eas...
EC2 > Target groups > microservice-tg
microservice-tg
Actions
Details
arn:aws:elasticloadbalancing:us-east-1:281712468655:targetgroup/microservice-tg/928066cce532f8bd
Target type
Instance
Protocol : Port
HTTP: 80
Protocol version
HTTP1
VPC
vpc-04a8d968cccd716d6
IP address type
IPv4
Load balancer
microservice-alb
1
Total targets
1
Healthy
0 Anomalous
0
Unhealthy
0
Unused
0
Initial
0
Draining
Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table below.


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LoadBalancer:loadBalancerArn=arn:aws:elasticloadbalancing:us-e...
EC2 > Load balancers > microservice-alb
microservice-alb
Actions
Details
Load balancer type
Application
Status
Active
VPC
vpc-04a8d968cccd716d6
Load balancer IP address type
IPv4
Scheme
Internet-facing
Hosted zone
Z35SXDOTRQ7X7K
Availability Zones
subnet-03dd5e41c656f63ad us-east-1a (use1-az1)
subnet-02393bb4fde726298 us-east-1b (use1-az2)
Date created
November 3, 2026, 18:29 (UTC+05:30)
Load balancer ARN
arn:aws:elasticloadbalancing:us-east-1:281712468655:loadbalancer/app/microservice-alb/696729a0a2103886
DNS name Info
microservice-alb-574490697.us-east-1.elb.amazonaws.com (A Record)


Ask Gemini
us-east-1.console.aws.amazon.com/ecr/repositories/private/281712468655/microservice-repository/_/image/sha256:b012a7be6597...
Relaunch to update
Search
[Alt+S]
United States (N. Virginia)
VA (281712468655)
VA
Amazon ECR > Private registry > Repositories > microservice-repository > sha256:b012a7be659709940d9ed66c3906c0fcea15e...
Amazon Elastic Container Registry
Private registry
Repositories
microservice-repository
Features & Settings
Public registry
Repositories
Settings
ECR public gallery
Amazon ECS
Amazon EKS
Image Index
Archive
Details
Image tags
1.0
URI
281712468655.dkr.ecr.us-east-1.amazonaws.com/microservice-repository:1.0
Digest
sha256:b012a7be659709940d9ed66c3906c0fcea15eb0711c2b7fcb2d4b7bebaf168e2
General information
Artifact type
Image Index
Repository
microservice-repository
Pushed at
November 03, 2026, 18:38:06 (UTC+05.5)
Last recorded pull time
November 03, 2026, 18:40:15 (UTC+05.5)
Size (MB)
26.34
Image status
Active
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
24°
ENG
IN
20:56
03-11-2026


Not secure
microservice-alb-574490697.us-east-1.elb.amazonaws.com
Relaunch to update
Welcome to the Microservice Application!
Successfully deployed using Docker and Terraform on AWS.
ENG
IN
18:44
03-11-2026